

APPENDIX

This appendix summarizes the prompt templates used in EMERGE. Following the style of the reference appendix, each template is denoted by P_s and is grouped by its role in the memory-augmented multi-agent optimization pipeline. Dynamic fields are written in braces, e.g., $\{problem_description\}$.

A. Task Classification and Memory Retrieval

1) *Prompt Template of MRTA Category Selection:* The prompt template in MRTA category selection: **Ps1**

You are a classifier that categorizes task descriptions according to the following three rules, and directly outputs one of the 8 corresponding classes (ST_SR_IA, ST_MR_IA, MT_SR_IA, MT_MR_IA, ST_SR_TA, ST_MR_TA, MT_SR_TA, MT_MR_TA):

Classification Rules:

1. ST/MT: Determine whether single robot execute multiple tasks. If yes, output MT; otherwise, output ST.
2. SR/MR: Determine whether there are any tasks that require multiple robots to collaborate for completion. If yes, output MR; otherwise, output SR.
3. IA/TA: Determine whether task planning considers timesteps. If yes, output TA; otherwise, output IA.

Class Explanation (combined in order):

- First part: ST (Single Task) or MT (Multi-Task)
- Second part: SR (Single Robot) or MR (Multi-Robot)
- Third part: IA (Instantaneous Assignment) or TA (Time-Extended Assignment)

Carefully read the task description, apply the three rules in sequence, and then output only the final class name without any additional text.

Task Description:
 $\{problem_description\}$

2) *Prompt Template of Memory Summary Evolution:* The prompt template in memory summary evolution: **Ps2**

You are an experienced work review expert, proficient in summarization, comparison, and iterative writing. Your core capability is: by comparing old summary and new summary, extracting effective patterns, preserving the essence, and integrating new insights to generate a more mature and more instructive iterative version of the work summary.

You will receive two inputs:

1- $\{current_summary\}$
 2- $\{original_summary\}$

Your Task: Revise and upgrade the Original Work Summary by integrating the new insights and lessons from the Current Work Problem and Current Work Summary.

Output Rules:

- 1-Output only the final, revised summary text. No explanations.
- 2-Preserve the original's core structure, successful conclusions, and its overall tone/style.

B. Multi-Agent Solver Prompts

1) *Prompt Template of Parser:* The prompt template for the Parser agent P_{Par} : **Ps3**

You are an expert in identifying and extracting key decision variables from the problem description to support accurate modeling.
 As the Parser, your task is to analyze the problem description and extract relevant variables, constraints, and objectives.
 Use your domain expertise to ensure these elements are accurately defined and suitable for formulating a solvable LP or MIP model.
 Specifically, ensure constraints use only $\leq$, $\geq$, or $=$ operators (avoid $>$ or $<$).

{example_context}

Review the problem description
 {problem_description}
 and the comments from other experts
 {comments_text}
 then provide the extracted variables, constraints, and objectives along with their definitions.

When retrieved demonstrations are available, {example_context} is instantiated by one of the following two templates.

this is the specific example : {example_str}

If the following information describes a successful summary or unsuccessful analysis , you may refer to it for context.: {example_str}

The example string used by this stage follows the format below.

To guide your work, study the following examples carefully. Each example demonstrates the complete thought process from problem description to structured extraction:

```
---
### EXAMPLE {i} ###
Problem Description:
{example_problem}
Example Output:
{example_output}
---
***End of Examples***
```

2) Prompt Template of Modeler: The prompt template for the Modeler agent P_{Mod} : **Ps4**

You are a modeling expert in Operations Research and Optimization, specializing in Mixed-Integer Programming (MIP). Your task is to construct a mathematical optimization model based on the problem description and insights provided by other reasoning stages. Leverage your expertise to formulate the optimization objectives and constraints, ensuring the model is comprehensive and suitable for solving the given production challenge. Please integrate all inputs and provide a well-defined model that aligns with operational research principles.

Now the origin problem is as follow:

{problem_description}

And the comments from other reasoning stages are as follow:

{comments_text}

Give your MIP model of this problem. Additionally, please note that your model needs to be a solvable linear programming model or a mixed-integer programming model. This also means that the expressions of the constraint conditions can only be

equal to, greater than or equal to, or less than or equal to (> or < are not allowed to appear and should be replaced to be \geq or \leq).

Your output format should be a JSON like this:

```
{
  "VARIABLES": "A mathematical description about variables",
  "CONSTRAINTS": "A mathematical description about constraints",
  "OBJECTIVE": "A mathematical description about objective"
}
```

3) Prompt Template of Developer: The prompt template for the Developer agent P_{Dev} : **Ps5**

You are a Python programmer specializing in operations research and optimization, with expertise in implementing and solving mathematical problems using Gurobi. Your main responsibility is to write, debug, and optimize code that transforms given optimization formulations into executable solutions using Gurobi. Ensure your implementation aligns with the problem objectives and constraints, and deliver clean, efficient, and well-documented code. While Gurobi is your primary tool, knowledge of related libraries such as NumPy, SciPy, or PuLP can support additional functionality or preprocessing tasks. Your goal is to provide robust, solver-ready code that effectively addresses the optimization problem.

As the Developer, your core task is to translate the parsed task information and mathematical formulation into executable solver code.

Crucially, you MUST ensure all constraints are expressed using only the operators $\leq$, $\geq$, or $=$; the use of strict inequalities ($>$ or $<$) is prohibited for linear programming formulations.

{example_str}

Analyze the problem step by step based on the provided description

{problem_description}

and the comments from other reasoning stages

{comments_text}.

Using the starter code template {code_example} write a Python function that strictly follows the given format and defines a solvable linear programming (LP) or mixed-integer programming (MIP) model.

Include only the necessary import statements and ensure no external test code is provided. Deliver clean, efficient, and well-structured code that aligns with the problem requirements.

When retrieved code demonstrations are used, {example_str} is instantiated as follows.

To guide your work, study the following examples carefully. Each example demonstrates the complete thought process from problem description to structured extraction:

EXAMPLE {i}

Problem Description:

{example_problem}

Example Code Implementation:

{example_code}

End of Examples

C. Memory Construction and Diagnosis Prompts

1) Prompt Template of Successful Case Summarization: The prompt template in successful case summarization: **Ps6**

You are an efficient Model Refinement Expert, skilled at condensing complex optimization models into their core essentials.

Review the following problem and model. Briefly summarize the problem description, optimization variables, constraints, objective function, and Precautions (1-3 sentences each).

this is the original problem: {problem_description}

this is the model file: {identify_text}

Output Constraints:

One-Sentence Principle: Each item (Variables, Constraints, Objective) must be expressed in exactly one sentence.

Professional Language: Use general mathematical terminology (e.g., binary variables, linear constraints, etc.) where appropriate, but do not use any mathematical formulas.

Minimalist Format: Do not include any redundant explanations. Output the concise summary directly using the following structure:

[Problem Summary]: [Insert summary here]

[Optimization Variables]: [Insert summary here]

[Optimization Constraints]: [Insert summary here]

[Optimization Objective]: [Insert summary here]

[Precautions]: [insert precautions here]

2) Prompt Template of Failed Case Diagnosis: The prompt template in failed case diagnosis: Ps7

You are an efficient Model Diagnosis Expert, skilled at quickly identifying core logical errors in optimization models.

Please analyze the following original problem and a model known to be incorrect. Your task is to directly and concisely explain why this modeling approach is wrong.

this is the original problem: {problem_description}

this is the model file: {identify_text}

Output Requirements:

Focus on the Root Cause: Analyze the fundamental logical error or key assumption bias in the modeling, not surface-level or syntax issues.

Fixed Structure: Use only the following structure for your output, with no introductory summary.

Concise Language: Explain each part in 1-2 sentences.

Output Structure (Strictly Follow):

[Error Cause Analysis]

[Core Error]: [State the most fundamental error in 1-2 sentences]

[Specific Analysis]: [Briefly explain the contradiction between the model logic and the problem requirements]

[Potential Consequences]: [The typical result of such an error, e.g., meaningless solution, infeasible model, etc.]

D. Baseline Prompt Templates

1) Prompt Template of Direct Standard Prompting: The prompt template in direct standard prompting: Ps8

You are a Python programmer specializing in operations research and optimization. Your expertise in using the Gurobi solver is essential for this task. You are given a specific optimization problem and must use Gurobi to solve it.

Please develop an efficient Python program that utilizes the Gurobi solver to address the following problem:

{problem}

Please provide the Python code directly, ensuring that Gurobi is the sole solver used in the solution.

2) *Prompt Template of Chain-of-Thought Baseline:* The prompt template in chain-of-thought baseline: **Ps9**

You are a Python programmer specializing in operations research and optimization. Your expertise in using the Gurobi solver is essential for this task. You are given a specific optimization problem and must use Gurobi to solve it. You are given a specific problem. You aim to develop an efficient Python program that addresses the given problem. Now the origin problem is as follow: {problem_description} Let's analyse the problem step by step, and then give your Python code. Here is a starter code: {code_example}

3) *Prompt Template of PHP Baseline:* The prompt template in PHP baseline: **Ps10**

You are a Python programmer in the field of operations research and optimization. Your expertise in using the Gurobi solver is essential for this task. You are given a specific optimization problem and must use Gurobi to solve it. You are given a specific problem. You aim to develop an efficient Python program that addresses the given problem. Now the origin problem is as follow: {problem_description} Let's analyse the problem step by step, and then give your Python code. Here is a starter code: {code_example}

The code looks like as following: {history_answer}

You need to check that the code is correct and can be executed directly.

The main paper introduces a taxonomy-driven benchmark and an execution-based evaluation protocol. This section provides additional details that are useful for reproducing the reported experiments and interpreting the error analysis.

E. MRTA Category Definitions

The benchmark follows three binary dimensions from the MRTA taxonomy: robot capability, task requirement, and assignment temporality. Their combinations produce the eight categories used throughout the experiments.

Single-task (ST) vs. multi-task (MT) robots. In ST settings, each robot is assigned to at most one task in the relevant assignment decision. In MT settings, a robot may serve multiple tasks across the assignment process, while time-extended cases still enforce the per-time-step execution constraints specified by the instance.

Single-robot (SR) vs. multi-robot (MR) tasks. In SR settings, each task requires exactly one robot. In MR settings, at least one task requires multiple robots to execute the same task, which introduces coalition or synchronization constraints.

Instantaneous assignment (IA) vs. time-extended assignment (TA). IA settings solve a static allocation problem without temporal scheduling. TA settings require assignments to satisfy task time windows and per-step robot availability constraints.

TABLE I
MRTA CATEGORY DEFINITIONS USED IN THE BENCHMARK.

Category	Robot capability	Task requirement	Temporality
ST/SR/IA	Single-task robots	Single-robot tasks	Instantaneous
ST/SR/TA	Single-task robots	Single-robot tasks	Time-extended
ST/MR/IA	Single-task robots	Multi-robot tasks	Instantaneous
ST/MR/TA	Single-task robots	Multi-robot tasks	Time-extended
MT/SR/IA	Multi-task robots	Single-robot tasks	Instantaneous
MT/SR/TA	Multi-task robots	Single-robot tasks	Time-extended
MT/MR/IA	Multi-task robots	Multi-robot tasks	Instantaneous
MT/MR/TA	Multi-task robots	Multi-robot tasks	Time-extended

F. Benchmark Instance Schema

Each benchmark instance is stored as one problem folder and contains three files: a natural-language description, a starter function template, and a JSON test sample. The natural-language description provides the task semantics and allocation requirements. The starter template fixes the required function name, input variables, and output format. The JSON file provides executable test inputs and ground-truth outputs used by the evaluator.

TABLE II
FIELDS COMMONLY USED BY BENCHMARK INSTANCES.

Field	Meaning
RobotPositions	2-D coordinates of robots.
TaskPositions	2-D coordinates of tasks.
NumRobots	Number of robots in the instance.
NumTasks	Number of tasks in the instance.
TaskRequirements	Number of robots required by each task, when applicable.
TimeSteps	Planning horizon length for time-extended settings.
EarliestStep	Earliest feasible execution step for each task.
LatestStep	Latest feasible execution step for each task.
output	Ground-truth allocation structure or optimal objective value.

The output format depends on the category. Some instantaneous assignment settings return explicit robot-task allocation structures, whereas temporal optimization settings often return the optimal total Euclidean travel distance. In all cases, success requires the generated program to follow the starter function signature and return an output that satisfies the category-specific constraints.

G. Execution-Based Success Criterion

For each benchmark instance, the model produces Python solver code. The code is extracted, executed with the instance-specific test sample, and checked by an automatic evaluator. A trial is counted as successful only if all of the following conditions hold:

- 1) the generated Python program can be parsed and executed without syntax or runtime errors;
- 2) the program preserves the required function name and input variables defined in the starter template;
- 3) the returned allocation or objective value matches the ground-truth output within the evaluator tolerance;
- 4) the returned result satisfies all MRTA constraints, including robot capacity, task coverage, multi-robot task requirements, and temporal windows when applicable.

H. Ablation Settings

The ablation study in the main paper isolates the contribution of each memory component. Table III summarizes the exact meaning of each variant.

TABLE III
DEFINITIONS OF ABLATION VARIANTS.

Variant	Removed or disabled component
w/o Mem	Disable memory retrieval and memory-conditioned prompting.
w/o Evo	Disable abstract-memory evolution after task execution.
w/o Forget	Disable access-frequency-based memory retention.
w/o Corr	Disable execution-feedback-based correction and diagnosis.
Ours	Use retrieval, abstract evolution, retention, and correction.

I. Error-Type Annotation

The error analysis in the main paper classifies each failed instance according to the earliest identifiable failure source. The categories are defined as follows.

TABLE IV
ERROR CATEGORIES USED IN THE FAILURE ANALYSIS.

Error type	Definition
Optimization objective	The solver optimizes a quantity inconsistent with the task goal.
Constraint condition	The solver omits, weakens, or misstates a required MRTA constraint.
Syntax	The generated Python code cannot be parsed or executed.
Output format	The solver computes a plausible result but returns it in the wrong format.

When multiple errors appear in the same generated solution, the label is assigned according to the earliest failure source in the solving pipeline. For example, if the code is executable but encodes an incorrect time-window constraint, the failure is counted as a constraint-condition error rather than an output-format error.

J. Memory Entry Structure

Each category-specific memory buffer contains one abstract memory and a bounded set of instance memories. The abstract memory stores category-level modeling knowledge, while each instance memory stores a concrete task-solving episode:

$$m_{ij} = \{\mathbf{h}_{ij}, u_{ij}, z_{ij}, \Pi_{ij}, c_{ij}, e_{ij}, a_{ij}, s_{ij}\}.$$

Here, u_{ij} is the task description, $\mathbf{h}_{ij}$ is its text embedding, z_{ij} is the parsed task representation, Π_{ij} is the optimization formulation, c_{ij} is the generated solver program, e_{ij} is execution feedback, a_{ij} is the returned allocation or objective value, and s_{ij} is the access frequency used for retention.

K. Retrieval and Retention

For a new task, EMERGE first predicts its MRTA category and then retrieves within the corresponding category buffer. Text embeddings are computed using `all-MiniLM-L6-v2`. Instance memories are ranked by cosine similarity to the new task embedding, and the top- K entries are retrieved together with the category-level abstract memory. Whenever an instance memory is retrieved, its access frequency is incremented. If the buffer exceeds its capacity N_i , entries with higher access frequency are retained.

This section lists the concrete implementation choices needed to reproduce the reported experiments. Provider-specific API keys, proxy addresses, and private endpoints are intentionally omitted; they should be replaced by any OpenAI-compatible LLM endpoint that supports the selected backbone model.

L. Software Environment

The implementation is written in Python and uses LangChain to call OpenAI-compatible chat models. The generated solver programs use Gurobi as the optimization backend. A working Gurobi installation and license are therefore required.

TABLE V
CORE SOFTWARE DEPENDENCIES USED FOR REPRODUCTION.

Dependency	Version or role
Python	3.10 or newer recommended
gurobipy	13.0.0
langchain	0.1.11
openai	2.9.0
sentence-transformers	5.1.2
transformers	4.57.3
torch	2.9.1
numpy	1.26.4
scipy	1.15.3
scikit-learn	1.7.2

The full dependency list is provided by `requirements.txt`. To reproduce the experiments, install the dependencies and configure the LLM backend before running any experiment script:

```
pip install -r requirements.txt
```

M. Repository Layout

The following files and directories are used in the experimental pipeline.

```
agents/ # Parser, Modeler, and Developer prompts
agentic_memory_rb/ # memory storage, retrieval, scoring, and evolution
baseline/ # Standard, CoT, and PHP baselines
dataset/ # eight MRTA categories, 25 instances each
scripts/ablation/ # ablation experiment helpers
main.py # multi-agent reasoning loop
run_exp.py # single-instance experiment entry point
run_exp_batch_insist.py # full benchmark runner
test_generated_code.py # execution-based evaluator
utils.py # problem loading and code extraction helpers
```

Each problem folder has the following structure:

```
dataset/<MRTA_CATEGORY>/prob_k/
description.txt # natural-language task description
code_example.py # required function signature and starter template
sample.json # executable input and ground-truth output
```

N. Model and Decoding Settings

All LLM calls are made with temperature 0. The default model name in the experiment entry point is `deepseek-ai/DeepSeek-V3`. The cross-model study in the main paper uses the same pipeline while replacing only the LLM backbone. The tested backbones are `DeepSeek-V3`, `Claude-3.5`, `Gemini-2.5-Flash`, `Gemini-2.5-Pro`, `GPT-4o`, `DeepSeek-V2.5`, and `Qwen2.5-VL`.

For reproducibility, the LLM endpoint should be configured in the files that instantiate `ChatOpenAI`. The scientific settings are independent of the specific endpoint URL or API key, which are environment-specific and are not part of the paper artifact.

O. Single-Instance Execution

A single instance can be evaluated with `run_exp.py`. The command below runs the full EMERGE configuration on one instance.

```
python run_exp.py \
  --dataset ST_SR_TA \
  --problem prob_0 \
  --algorithm coe \
  --model deepseek-ai/DeepSeek-V3 \
  --use true \
  --useab true \
  --record true \
  --evolve true \
  --forget true \
  --check true \
  --max_collaborate_nums 5
```

The main command-line arguments are summarized below.

TABLE VI
MAIN `RUN_EXP.PY` ARGUMENTS.

Argument	Default	Meaning
<code>--algorithm</code>	coe	Use EMERGE/chain-of-experts or a baseline.
<code>--model</code>	deepseek-ai/DeepSeek-V3	LLM backbone name.
<code>--use</code>	true	Enable instance-level memory retrieval.
<code>--useab</code>	true	Enable abstract memory retrieval.
<code>--record</code>	true	Record new memory after execution.
<code>--evolve</code>	true	Update abstract memory summaries.
<code>--forget</code>	true	Enable score-based memory retention.
<code>--check</code>	true	Use execution feedback for memory update.
<code>--max_collaborate_nums</code>	5	Maximum reasoning-stage consultation rounds.

P. Full Benchmark Execution

The full benchmark contains 200 instances: eight MRTA categories and 25 instances per category. The categories are evaluated in the order used by `run_exp_batch_insist.py`:

```
MT_MR_TA, MT_SR_TA, ST_SR_IA, MT_MR_IA,
ST_SR_TA, ST_MR_TA, ST_MR_IA, MT_SR_IA
```

The full benchmark runner can be launched as follows:

```
python run_exp_batch_insist.py
```

The runner sets `num_problems_per_dataset=25` and `random_order=False`. It requeues a task if the experiment subprocess fails or times out; this retry behavior is used to handle infrastructure-level failures. The reported success rate is still determined only by the evaluator result for each benchmark instance.

Q. Baseline Execution

The same `run_exp.py` entry point is used for the baselines reported in the paper by changing `--algorithm`. The relevant names are:

```
standard
chain_of_thought # alias: cot
php
```

For example, the standard baseline can be run as follows:

```
python run_exp.py \
  --dataset ST_SR_TA \
  --problem prob_0 \
  --algorithm standard \
  --model deepseek-ai/DeepSeek-V3
```

R. Ablation Execution

Ablation scripts are provided under `scripts/ablation/`. They patch the batch runner in a temporary copy and execute the selected configuration. The helper scripts in the repository currently set `num_problems=10` per category for quick ablation runs; setting `num_problems=25` reproduces the full 200-instance ablation setting.

```
python scripts/ablation/run_no_memory.py
python scripts/ablation/run_no_evolve.py
python scripts/ablation/run_no_forget.py
python scripts/ablation/run_no_check.py
```

The exact flags for each ablation are:

TABLE VII
ABLATION FLAGS USED BY THE EXPERIMENT SCRIPTS.

Variant	use	useab	record	evolve	forget	check
w/o Mem	false	false	false	false	false	false
w/o Evo	true	true	true	false	true	true
w/o Forget	true	true	true	true	false	true
w/o Corr	true	true	true	true	true	false
Ours	true	true	true	true	true	true

S. Memory Hyperparameters

The memory system is initialized with the sentence-transformer encoder `all-MiniLM-L6-v2`. The following settings are used in `run_exp.py`.

TABLE VIII
MEMORY-RELATED HYPERPARAMETERS.

Parameter	Value	Description
Specific-memory retrieval K	1	Top similar instance memory in same category.
Specific-memory tolerance	0	Retrieve only from the routed category.
Abstract-memory target number	2	Number of abstract memories retrieved.
Abstract-memory tolerance	2	Category-distance tolerance for abstract retrieval.
Abstract memory per category	1	Initial abstract-memory count per category.
Memory score increment	2	Score added when a new accepted note is stored.
Specific-memory retention size	5	Maximum retained specific memories per category.

During the main experiments, the dataset name is used as the task category when running `run_exp.py`. The category-selection prompt is included in the appendix because it is implemented in the codebase and can be used when category labels are unavailable, but the benchmark experiments route each instance by its known dataset category.

T. Multi-Agent Reasoning Procedure

For EMERGE, the multi-agent reasoning process contains three specialized stages: Parser, Modeler, and Developer. The system runs up to five consultation rounds, and the generated answer is post-processed by extracting fenced Python code blocks. If no code block is found, the raw answer is used as code. A small deterministic post-processing rule replaces `inrange` with `in range` before execution.

U. Automatic Evaluator

The evaluator imports the generated file as `generated_code.py`, reloads it, and calls the function whose name matches the problem folder, e.g., `prob_0`. If the file cannot be imported or the function cannot be loaded, the result is `COMPILE_ERROR`. If the function raises an exception on any test sample, the result is `RUNTIME_ERROR`. Otherwise, the returned output is compared with the ground truth in `sample.json`.

For scalar numeric outputs, the evaluator accepts the result if

$$|\hat{y} - y| < 0.1.$$

For non-scalar or structured outputs, exact equality is required. A problem is counted as accepted only if all test samples pass. The result labels are: `ACCEPT`, `WRONG_ANSWER`, `RUNTIME_ERROR`, and `COMPILE_ERROR`.

V. Generated Artifacts and Logs

Each run creates a folder under `log/` with a timestamped name. The main artifacts are:

```
<problem>_original_answer.txt # raw LLM output
<problem>_generated_code.py # extracted executable code
<problem>_analysis.txt # reasoning-stage trace used for memory
<problem>_using_memory.txt # retrieved memory ids and memory levels
<problem>_test_log.txt # evaluator output and pass/fail status
<problem>_summary.txt # success summary or failure diagnosis
<problem>_renew_summary.txt # evolved abstract memory summary
```

Persistent memories are stored under `agentic_memory_rb/memory/` or the configured memory directory. For reproducible comparisons, the same initial memory state should be used across methods or ablations.

W. Reproduction Checklist

To reproduce the reported results, use the following checklist.

- 1) Install dependencies from `requirements.txt` and verify the Gurobi license.
- 2) Configure the OpenAI-compatible LLM endpoint and set temperature to 0.
- 3) Ensure all eight dataset folders contain 25 instances each.
- 4) Use `run_exp.py` for single-instance runs and `run_exp_batch_insisit.py` for the full benchmark.
- 5) Use the same memory initialization and memory flags for all methods being compared.
- 6) Compute success rates from evaluator labels, counting only `ACCEPT` as successful.
- 7) For error analysis, annotate failed instances by the earliest failure source using the taxonomy in Table IV.